DSA & Advanced DSA Roadmap
Phase 14 — Graphs

Learning Objectives

By the end of this phase, learners will be able to:

- Represent a graph using an adjacency matrix and an adjacency list, and explain the space/time trade-offs between them.
- Apply BFS and DFS to general graphs (not just trees), correctly handling cycles via a visited set.
- Detect cycles in both directed and undirected graphs, using the technique appropriate to each (coloring/recursion-stack tracking for directed; parent-tracking or Union Find for undirected).
- Determine whether a graph is bipartite, and produce a valid topological ordering of a directed acyclic graph.
- Implement Union Find (DSU) with path compression and union by rank, and apply it to connectivity and cycle-detection problems.
- Solve Easy to Medium level graph problems independently, applying Dijkstra's algorithm to weighted shortest-path problems that would otherwise require exhaustive search.

Implementation & Internals

Core Concepts

- Adjacency Matrix — an n×n grid where cell (i, j) indicates an edge between i and j; O(1) edge lookup but O(n²) space, wasteful for sparse graphs.
- Adjacency List — each node stores a list of its neighbors; O(V + E) space, the standard choice for most graph problems.
- Directed Graphs — edges have a direction (u → v does not imply v → u); relevant for prerequisite/dependency problems.
- Undirected Graphs — edges are symmetric; relevant for connectivity and bipartiteness problems.
- Weighted Graphs — each edge carries a cost, required for shortest-path algorithms like Dijkstra's.

Implementation Exercises

- Build a graph class from scratch supporting both adjacency matrix and adjacency list representations, and implement a conversion function between them.
- Implement generic BFS and DFS traversal functions that work on any adjacency-list-represented graph, returning the visit order.
- Implement Union Find from scratch with both path compression and union by rank, and demonstrate the near-O(1) amortized cost of find/union after several operations.

Pattern Mastery

Pattern 1: BFS

Concepts

- Level-by-level graph traversal using a queue, extended from trees (Phase 11) to general graphs where a visited set is now required to avoid infinite loops on cycles.
- Using BFS to clone a graph by mapping original nodes to copies as they're first discovered.

Practice Problems

- Given a reference node in a connected undirected graph, return a deep copy (clone) of the graph.

LeetCode

- Clone Graph — leetcode.com/problems/clone-graph

Pattern 2: DFS

Concepts

- Depth-first graph traversal using recursion or an explicit stack, again requiring a visited set on general graphs.
- Using DFS to explore and count separate groups of connected land cells in a grid, the most common 'graph problem disguised as a grid problem' shape.

Practice Problems

- Given a 2D grid of '1's (land) and '0's (water), count the number of islands, where an island is a group of connected '1's.

LeetCode

- Number of Islands — leetcode.com/problems/number-of-islands

Note: this is the problem reserved back in Phase 5, where it was deliberately excluded from the BFS Foundation pattern so it could serve as this phase's canonical connected-components-via-DFS example instead.

Pattern 3: Connected Components

Concepts

- Counting or grouping nodes that are reachable from each other, using repeated BFS/DFS (one traversal per unvisited node) or Union Find.
- Recognizing 'connected components' problems even when not phrased graph-first, e.g. when given an adjacency matrix of cities directly rather than an edge list.

Practice Problems

- Given an n×n adjacency matrix of cities, determine the number of provinces (groups of directly or indirectly connected cities).

LeetCode

- Number of Provinces — leetcode.com/problems/number-of-provinces

Pattern 4: Cycle Detection

Concepts

- In a directed graph, tracking three node states (unvisited, in-progress, fully processed) during DFS; finding an edge back to an in-progress node signals a cycle.
- In an undirected graph, using Union Find: if two nodes being connected by an edge are already in the same component, that edge creates a cycle.

Practice Problems

- Given n courses and a list of prerequisite pairs, determine whether it's possible to finish all courses (i.e. the prerequisite graph has no cycle).
- Given a graph that started as a tree and had one extra edge added, find that edge using Union Find.

LeetCode

- Course Schedule — leetcode.com/problems/course-schedule
- Redundant Connection — leetcode.com/problems/redundant-connection

Pattern 5: Bipartite Graphs

Concepts

- Two-coloring a graph via BFS or DFS: assign a node one color, assign every neighbor the opposite color, and fail if a conflict is ever found.
- Recognizing that a graph is bipartite if and only if it has no odd-length cycle, connecting back to the Cycle Detection pattern.

Practice Problems

- Given an undirected graph, determine whether its nodes can be split into two groups such that every edge connects nodes from different groups.

LeetCode

- Is Graph Bipartite? — leetcode.com/problems/is-graph-bipartite

Pattern 6: Topological Sort

Concepts

- Producing a linear ordering of a directed acyclic graph's nodes such that every edge points from an earlier node to a later one, using either DFS-based postorder reversal or Kahn's BFS-based in-degree algorithm.
- Recognizing that a valid topological order exists if and only if the graph has no cycle, directly reusing the Cycle Detection pattern's DFS coloring technique.

Practice Problems

- Given n courses and prerequisite pairs, return a valid order in which to take all courses, or an empty array if no such order exists.

LeetCode

- Course Schedule II — leetcode.com/problems/course-schedule-ii

Pattern 7: Union Find (DSU)

Concepts

- Maintaining a forest of disjoint sets where each set has a representative ('parent'); find determines which set a node belongs to, and union merges two sets.
- The two optimizations that make Union Find efficient in practice: path compression (flatten the tree on find) and union by rank/size (always attach the smaller tree under the larger one).

Practice Problems

- Reframe Redundant Connection and Number of Provinces (already solved above) explicitly using a from-scratch Union Find implementation, rather than DFS/BFS, to compare both approaches on the same problems.

No new problems for this pattern: Union Find is best learned by re-solving Redundant Connection and Number of Provinces (Patterns 3 and 4 above) with a from-scratch DSU instead of traversal-based code, directly comparing both techniques on identical inputs.

Pattern 8: Shortest Path

Concepts

- Dijkstra's algorithm: repeatedly extracting the closest unvisited node from a min-heap (Phase 12) and relaxing its neighbors' distances, guaranteed correct only when all edge weights are non-negative.
- Recognizing the relationship between BFS (shortest path in an unweighted graph, all edges 'cost' 1) and Dijkstra's algorithm (shortest path in a weighted graph, edges have varying cost).

Practice Problems

- Given a weighted directed graph and a source node, find the minimum time for a signal to reach every other node, or determine it's impossible.

LeetCode

- Network Delay Time — leetcode.com/problems/network-delay-time

Phase Assessment

Implementation Tasks

- Build a graph class supporting both adjacency matrix and list representations, with generic BFS and DFS traversal methods.
- Build Union Find from scratch with path compression and union by rank, then re-solve Redundant Connection and Number of Provinces using it instead of traversal-based code.
- Analyze and explain why Dijkstra's algorithm fails on graphs with negative edge weights, using a small counterexample graph.

Recommended LeetCode Target

Easy Problems

None assigned this phase — graph problems typically require enough structure (cycles, weights, multiple components) to be meaningful, which pushes most graph problems to Medium or above on LeetCode.

Medium Problems

BFS

- leetcode.com/problems/clone-graph

DFS / Connected Components

- leetcode.com/problems/number-of-islands
- leetcode.com/problems/number-of-provinces

Cycle Detection

- leetcode.com/problems/course-schedule
- leetcode.com/problems/redundant-connection

Bipartite Graphs

- leetcode.com/problems/is-graph-bipartite

Topological Sort

- leetcode.com/problems/course-schedule-ii

Shortest Path

- leetcode.com/problems/network-delay-time

On scope and dedup

This phase has 8 distinct problems covering 8 patterns, with no Easy tier and no Hard tier: graph problems cluster heavily at Medium difficulty on LeetCode because meaningful graph structure (cycles, multiple components, weighted edges) is what makes a problem interesting, and that same structure pushes difficulty up from Easy. Hard-tier graph problems (e.g. Alien Dictionary, Critical Connections) are deferred to Phase 17 (Competitive Programming) where they fit alongside other advanced techniques. The Union Find pattern intentionally reuses two problems already listed under Connected Components and Cycle Detection rather than introducing new ones, since the pedagogical goal is comparing techniques on identical problems, not covering more ground. None of the 8 problems overlaps with any problem used in Phases 0–13, and Number of Islands fulfills the reservation made in Phase 5.

Coding Challenges (Assessment Day)

- Number of Islands
- Course Schedule
- Course Schedule II
- Redundant Connection (solved twice: DFS-based, then Union-Find-based)
- Network Delay Time

Expected Outcome

Students should be able to:

- Represent a graph using both adjacency matrix and adjacency list, and implement BFS/DFS that correctly handle cycles.
- Recognize and apply all eight graph patterns: BFS, DFS, connected components, cycle detection, bipartite checking, topological sort, Union Find, and shortest path.
- Solve unseen graph problems using pattern-based thinking rather than memorized solutions.
- Choose between traversal-based and Union-Find-based approaches for connectivity problems, and explain the trade-offs of each.